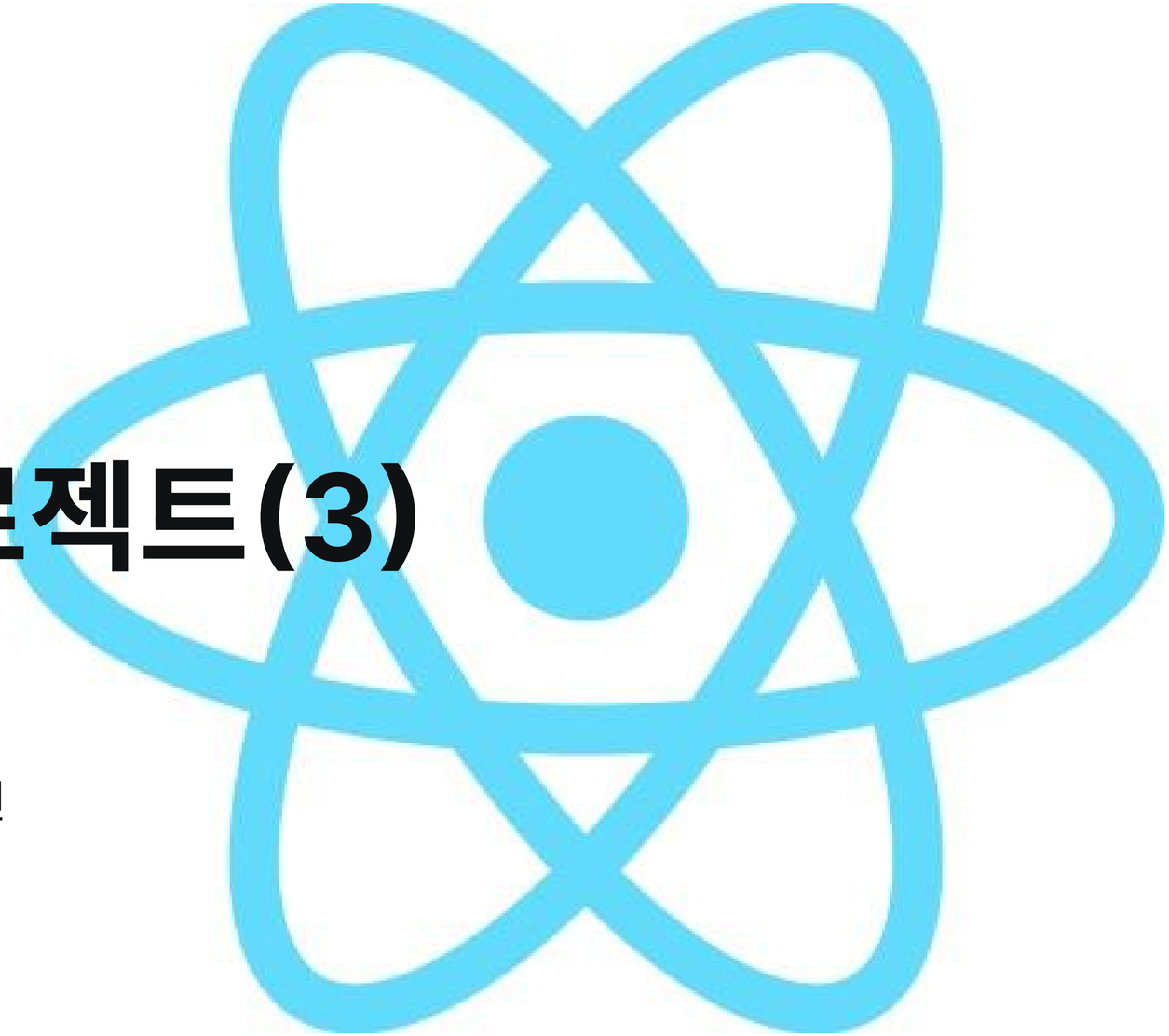
React 쇼핑몰 프로젝트(3)



EY 기업맞춤형 AI+X 융복합과정 보조강사 정승민

학습 안내

3일차 목차

01 **1교시 로그인 API와 JWT**

고객 테이블, 비밀번호 해시, 로그인 API와 JWT

02 **2교시 React 로그인 연결**

로그인 폼, 회원 확인, 로그아웃과 환영 문구

03 **3교시 주문 저장**

주문 API, 재고와 트랜잭션, 주문 성공 카드

04 **4교시 화면 이동과 회원 정보**

라우팅, 로그인 후 이동, 회원 정보와 메뉴 강조

App.css 전체 교체

src/App.css: 실습 전에 기존 내용 전체를 아래 코드로 교체

```
#root{width:100%;min-height:100vh;}.shop{width:min(100%,1120px);margin:0 auto;padding:48px 24px 80px;}.shop-header{margin-bottom:48px;text-align:center;}.shop-header p{margin:0 0
8px;color:#7c3aed;font-weight:700;}.shop-header h1{margin:0 0 10px;color:#1f2937;font-size:clamp(32px,4vw,48px);line-height:1.2;}.shop-header span{color:#6b7280;}.shop h2{margin:0 0
20px;color:#1f2937;font-size:24px;}.product-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(220px,1fr));gap:20px;}.product-card{display:flex;min-height:320px;flex-
direction:column;padding:20px;border:1px solid #e5e7eb;border-radius:16px;background:#fff;box-shadow:0 8px 24px rgba(31,41,55,0.04);}.product-emoji{display:grid;height:148px;place-
items:center;border-radius:12px;background:#f5f3ff;font-size:56px;}.product-category{margin:16px 0 4px;color:#6b7280;font-size:14px;}.product-card h3{margin:0 0 12px;font-size:18px;}.product-
price{margin:auto 0 16px;font-weight:700;}.product-card button{width:100%;padding:11px 16px;border:0;border-radius:9px;background:#6d28d9;color:#fff;font-weight:700;}.product-card
button:hover{background:#5b21b6;}@media (max-width:640px){.shop{padding:28px 16px 56px;}.shop-header{margin-bottom:36px;}.product-grid{grid-template-columns:1fr;}}:root{font-
family:"Pretendard","Noto Sans KR",system-ui,-apple-system,sans-serif;color:#1f2937;background:#f7f8fc;font-synthesis:none;text-rendering:optimizeLegibility;-webkit-font-smoothing:antialiased;}
*{box-sizing:border-box;}body{min-width:320px;margin:0;}button,input,select{font:inherit;}button{cursor:pointer;}.featured-section{margin-bottom:48px;}.featured-
product{display:grid;width:100%;grid-template-columns:80px minmax(0,1fr) auto auto;align-items:center;gap:18px;padding:20px 24px;border:1px solid #e6e2f0;border-radius:16px;background:#fff;box-
shadow:0 8px 24px rgba(31,41,55,0.05);}.featured-product div[role="img"]{display:grid;height:80px;place-items:center;border-radius:12px;background:#f5f3ff;font-size:42px;}.featured-product
h3{margin:0;font-size:18px;}.featured-product p{margin:0;font-weight:700;}.featured-product button{padding:11px 16px;border:0;border-radius:9px;background:#6d28d9;color:#fff;font-
weight:700;white-space:nowrap;}.featured-product button:hover{background:#5b21b6;}.filter-panel{display:grid;grid-template-columns:minmax(0,1fr) minmax(180px,240px) auto;align-
items:end;gap:14px;margin-bottom:16px;padding:18px;border:1px solid #e5e7eb;border-radius:14px;background:#fff;}.filter-field{display:flex;min-width:0;flex-direction:column;gap:7px;}.filter-
field label{font-size:14px;font-weight:700;}.filter-field input,.filter-field select{width:100%;min-height:42px;padding:9px 12px;border:1px solid #d1d5db;border-
radius:8px;background:#fff;color:#1f2937;}.filter-field input:focus,.filter-field select:focus{outline:2px solid #a78bfa;outline-offset:1px;}.reset-button{min-height:42px;padding:9px
16px;border:1px solid #d1d5db;border-radius:8px;background:#fff;color:#374151;}.result-count{margin:0 0 16px;color:#6b7280;}.empty-state{padding:36px 20px;border:1px dashed #c4b5fd;border-
radius:12px;background:#fff;text-align:center;}@media (max-width:640px){.featured-product{grid-template-columns:64px minmax(0,1fr);padding:16px;}.featured-product div[role="img"]
{height:64px;grid-row:1 / span 2;}.featured-product h3{grid-column:2;grid-row:1;}.featured-product p{grid-column:2;grid-row:2;}.featured-product button{grid-column:1 / -1;}.filter-panel{grid-
template-columns:1fr;}}.cart-section{margin-top:48px;}.cart-list{margin:0;padding:0;list-style:none;}.cart-item{display:flex;align-items:center;justify-content:space-
between;gap:16px;padding:16px 20px;border:1px solid #e5e7eb;border-radius:12px;background:#fff;}.cart-item + .cart-item{margin-top:10px;}.cart-item p{margin:6px 0 0;color:#6b7280;}.quantity-
controls{display:flex;align-items:center;gap:12px;white-space:nowrap;}.quantity-controls button{width:36px;height:36px;border:1px solid #d1d5db;border-radius:8px;background:#fff;}.cart-
total{margin:18px 0 0;text-align:right;font-size:20px;font-weight:700;}@media (max-width:640px){.cart-item{align-items:flex-start;flex-direction:column;}}.order-section{margin-top:48px;}.order-
layout{display:grid;grid-template-columns:repeat(2,minmax(0,1fr));gap:20px;}.order-form,.order-summary{padding:24px;border:1px solid #e5e7eb;border-radius:16px;background:#fff;}.order-form
label{display:block;margin:14px 0 6px;font-weight:700;}.order-form input{width:100%;padding:10px;border:1px solid #d1d5db;border-radius:8px;}.order-form input[aria-invalid="true"]{border-
color:#dc2626;}.order-form button{margin-top:20px;padding:11px 18px;border:0;border-radius:9px;background:#6d28d9;color:#fff;}.form-error{color:#b91c1c;}.form-success{color:#166534;}.order-
summary ul{padding:0;list-style:none;}.order-summary li{display:flex;justify-content:space-between;gap:16px;padding:10px 0;border-bottom:1px solid #e5e7eb;}.order-total{text-align:right;font-
weight:700;}@media (max-width:640px){.order-layout{grid-template-columns:1fr;}}.api-status{margin:16px 0 24px;padding:16px 20px;border-radius:12px;background:#f5f3ff;color:#4c1d95;}.api-
error{background:#fef2f2;color:#b91c1c;}.auth-section{margin:0 0 48px;padding:24px;border:1px solid #e5e7eb;border-radius:16px;background:#fff;}.auth-section h2{margin:0 0 16px;}.auth-
form{display:grid;gap:12px;max-width:420px;}.auth-form label{display:block;margin-bottom:6px;font-weight:700;}.auth-form input{width:100%;padding:10px;border:1px solid #d1d5db;border-
radius:8px;}.auth-form button,.auth-signed-in button{width:fit-content;padding:10px 18px;border:0;border-radius:8px;background:#6d28d9;color:#fff;font-weight:700;}.auth-form
button:disabled{opacity:0.6;cursor:wait;}.auth-signed-in{display:flex;align-items:center;gap:16px;flex-wrap:wrap;}.auth-signed-in p{margin:0;}.order-form
button:disabled{opacity:0.6;cursor:wait;}.shop-nav{display:flex;flex-wrap:wrap;justify-content:center;gap:0.5rem;margin:1rem 0 1.5rem;}.shop-nav a{padding:0.65rem 1rem;border:1px solid
#ded7ec;border-radius:999px;color:#372c4a;text-decoration:none;background:#fff;}.shop-nav a:hover,.shop-nav a:focus-visible,.shop-nav a.active{color:#fff;background:#6d28d9;border-
color:#6d28d9;}.shop-nav a:focus-visible{outline:2px solid #371080;outline-offset:2px;}.order-form .member-info-option{display:flex;align-items:center;gap:8px;cursor:pointer;}.order-form
.member-info-option input{width:18px;height:18px;margin:0;padding:0;accent-color:#6d28d9;}.member-info-help{margin:0 0 18px;color:#6b7280;font-size:14px;}.order-form input[readonly]
{background:#f5f3ff;}.welcome-message{padding:12px;background:#f5f3ff;color:#4c1d95;}.welcome-message.signed-in{background:#ecfdf5;color:#166534;}.order-success-card{margin-
top:20px;padding:16px;border:2px solid #166534;border-radius:12px;background:#ecfdf5;color:#166534;}.order-success-card h3{margin:0 0 12px;}.shop-nav a.active{background:#0f766e;border-
color:#0f766e;font-weight:700;text-decoration:underline;text-underline-offset:4px;}
```

로그인 API와 JWT 인증

미션 - 계정으로 로그인하고, 현재 로그인한 유저를 확인한다.

01 **확인 결과**

/login의 JWT와 /me의 고객 정보

02 **시간**

DB · 계정 22분 → JWT 8분 → API 15분 → 확인 5분

기본키와 외래키로 관계 연결

행은 기본키로 구분하고, 다른 테이블의 행은 외래키로 연결합니다.

customers

customer_id (PK)	이름
1	민수

← 참조

같은 ID

sales

sale_id (PK)	customer_id (FK)	상품 ID
101	1	2
102	1	3

PK: 중복 없이 한 행을 구분

고객 1번은 customers에서 한 명을 가리킵니다.

FK: 존재하는 ID만 연결

없는 고객 ID 99를 연결하면 오류가 납니다.

고객과 판매 테이블 만들기

DBeaver에서 shop 연결의 SQL 편집기에 작성하고 실행

```
CREATE TABLE public.customers (
  customer_id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  userid VARCHAR(50) NOT NULL UNIQUE,
  passwd TEXT NOT NULL
);

CREATE TABLE public.sales (
  sale_id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  customer_id INTEGER NOT NULL REFERENCES public.customers(customer_id),
  product_id INTEGER NOT NULL REFERENCES public.products(product_id),
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  total_price NUMERIC(10, 2) NOT NULL CHECK (total_price >= 0),
  sale_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

인증 패키지 설치

Windows CMD

package.json이 있는 프로젝트 폴더에서 실행

```
backend\.venv\Scripts\python.exe -m pip install -r backend\requirements.txt
```

Python 패키지 목록

backend/requirements.txt

새 파일 생성 후 전체 입력

필요한 패키지를 한 파일에 모으면 다른 컴퓨터에서도 같은 목록을 한 번에 설치할 수 있습니다.

```
fastapi
uvicorn
psycopg[binary]
python-dotenv
PyJWT
pwdlib[argon2]
```

JWT 비밀키 생성 및 설정

Windows CMD

생성한 값은 backend/.env에 직접 기록

```
backend\.venv\Scripts\python.exe -c "import secrets; print(secrets.token_urlsafe(48))"
```

backend/.env

2일차에 만든 .env 파일 맨 아래에 한 줄만 추가

```
SHOP_JWT_SECRET=앞에서_생성한_비밀키
```

실습 계정 생성 프로그램 (1/2)

backend/create_demo_user.py

새 파일 생성 후 전체 입력. 1/2

```
from getpass import getpass

from pwdlib import PasswordHash

from backend.main import get_connection

userid = input("실습 아이디: ").strip()
name = input("이름: ").strip()
email = input("이메일: ").strip()
password = getpass("실습 비밀번호(8자 이상): ")

if not userid or not name or not email or len(password) < 8:
    raise SystemExit("아이디·이름·이메일과 8자 이상의 비밀번호가 필요합니다.")

hashed_password = PasswordHash.recommended().hash(password)


with get_connection() as connection:
    with connection.cursor() as cursor:
```

실습 계정 생성 프로그램 (2/2)

backend/create_demo_user.py

새 파일 생성 후 전체 입력. 2/2

```
hashed_password = PasswordHash.recommended().hash(password)
with get_connection() as connection:
    with connection.cursor() as cursor:
        cursor.execute(
            """
            INSERT INTO public.customers (name, email, userid, passwd)
            VALUES (%s, %s, %s, %s)
            ON CONFLICT (userid) DO NOTHING
            RETURNING customer_id
            """,
            (name, email, userid, hashed_password),
        )
        created = cursor.fetchone()

print("계정 생성 완료" if created else "이미 있는 아이디입니다.")
```

실습 계정 생성 실행

Windows CMD

아이디, 이름, 이메일, 비밀번호를 차례대로 입력

```
backend\.venv\Scripts\python.exe -m backend.create_demo_user
```

비밀번호 해시와 JWT

01

비밀번호 원문

로그인할 때 서버로 보내
검증합니다.

02

Argon2 해시

DB에 저장합니다.
verify로 입력 비밀번호를
검증합니다.

03

JWT

서명된 인증 문자열입니다.
내용 자체가 암호화되는
것은 아닙니다.

04

sub와 exp

sub는 고객 ID,
exp는 만료 시각입니다.
비밀번호는 넣지 않습니다.

로그인 요청의 흐름

01

React

아이디와 비밀번호

→

02

POST /login

DB 고객, 해시 확인

→

03

JWT 발급

고객 ID와 만료

→

04

GET /me

Bearer로 현재 고객 확인

Argon2로 비밀번호를 해시하는 이유

DB에 비밀번호 원문을 그대로 저장하지 않기 위해 해시를 사용합니다.

비밀번호 입력

사용자가 입력한 비밀번호

→

Argon2 해시

해시 문자열을 생성
반복 추측에 계산 비용 부과

→

DB에 저장

customers.passwd에는
해시 문자열을 저장

로그인: 입력 비밀번호 + 저장된 해시 → verify → 성공 또는 실패

해시를 원문으로 되돌리지 않고 입력한 비밀번호가 맞는지 검증합니다.

로그인 API: 서버 코드

backend/main.py

기존 import 문 앞에 추가

```
from datetime import datetime, timedelta, timezone

import jwt
from fastapi import Depends, HTTPException
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from jwt.exceptions import InvalidTokenError
from pwdlib import PasswordHash
from pydantic import BaseModel

from fastapi.middleware.cors import CORSMiddleware
```

로그인 API: 서버 코드

backend/main.py

서버 코드, 이전 단계 11줄 뒤에 추가

```
load_dotenv(Path(__file__).with_name(".env"))
JWT_SECRET = os.environ["SHOP_JWT_SECRET"]
password_hasher = PasswordHash.recommended()
bearer_scheme = HTTPBearer(auto_error=False)

class LoginRequest(BaseModel):
    userid: str
    password: str


app = FastAPI()
```

로그인 API: 서버 코드 (1/5)

backend/main.py

서버 코드, 이전 단계 62줄 뒤에 추가. 1/5

```
        for row in rows
    ]


@app.post("/login")
def login(body: LoginRequest):
    with get_connection() as connection:
        with connection.cursor() as cursor:
            cursor.execute(
                """
                SELECT customer_id, userid, name, passwd
                FROM public.customers
                WHERE userid = %s
                """,
                (body.userid,),
            )
            customer = cursor.fetchone()


    if customer is None or not password_hasher.verify(
        body.password, customer["passwd"]
```

로그인 API: 서버 코드 (2/5)

backend/main.py

서버 코드, 이전 단계 62줄 뒤에 추가. 2/5

```
        customer = cursor.fetchone()
    if customer is None or not password_hasher.verify(
        body.password, customer["passwd"]
    ):
        raise HTTPException(
            status_code=401,
            detail="아이디 또는 비밀번호가 올바르지 않습니다.",
            headers={"WWW-Authenticate": "Bearer"},
        )

    expires = datetime.now(timezone.utc) + timedelta(minutes=30)
    token = jwt.encode(
        {"sub": str(customer["customer_id"]), "exp": expires},
        JWT_SECRET,
        algorithm="HS256",
    )
    return {"access_token": token, "token_type": "bearer"}


def get_current_customer(
```

로그인 API: 서버 코드 (3/5)

backend/main.py

서버 코드, 이전 단계 62줄 뒤에 추가. 3/5

```
    )
    return {"access_token": token, "token_type": "bearer"}

def get_current_customer(
    credentials: HTTPAuthorizationCredentials | None = Depends(bearer_scheme),
):
    unauthorized = HTTPException(
        status_code=401,
        detail="인증이 필요합니다.",
        headers={"WWW-Authenticate": "Bearer"},
    )

    if credentials is None:
        raise unauthorized

    try:
        payload = jwt.decode(
            credentials.credentials,

          JWT_SECRET,
          algorithms=["HS256"],
```

로그인 API: 서버 코드 (4/5)

backend/main.py

서버 코드, 이전 단계 62줄 뒤에 추가. 4/5

```
        payload = jwt.decode(
            credentials.credentials,
                JWT_SECRET,
                algorithms=["HS256"],
        )
        customer_id = int(payload.get("sub", ""))
    except (InvalidTokenError, TypeError, ValueError):
        raise unauthorized

    with get_connection() as connection:
        with connection.cursor() as cursor:
            cursor.execute(
                """
                SELECT customer_id, userid, name, email
                FROM public.customers
                WHERE customer_id = %s
                """,
                (customer_id,),

            )
            customer = cursor.fetchone()
```

로그인 API: 서버 코드 (5/5)

backend/main.py

서버 코드, 이전 단계 62줄 뒤에 추가. 5/5

```
            """,
            (customer_id,),
        )
        customer = cursor.fetchone()

    if customer is None:
        raise unauthorized
    return customer

@app.get("/me")
def read_me(customer=Depends(get_current_customer)):
    return customer
```

FastAPI 서버 실행

접속 주소

127.0.0.1:8000/health
/docs에서 API 문서 확인

터미널: Windows CMD

프로젝트 폴더에서 실행, 이 터미널은 켜 둠

```
backend\.venv\Scripts\python.exe -m uvicorn backend.main:app --reload
```

1교시 핵심 정리

고객과 판매	기본키와 외래키로 실제 고객의 행을 연결
비밀번호 해시	Argon2 해시를 저장하고 로그인 시 입력값 검증
POST /login	아이디와 비밀번호를 확인하고 JWT 발급
GET /me	JWT의 서명과 만료를 검증하고 고객 정보 반환

React에 로그인 연결하기

미션 - 로그인 화면을 연결하고, 회원 이름과 로그아웃을 표시한다.

01 **완성 화면**

로그인 폼, 회원 이름과 환영 문구

02 **시간**

상태 8분 → 폼 10분 → 요청 17분 → 확인 10분 → 응용 5분

React의 로그인 상태

App이 로그인 상태를 기억

`loginForm`
아이디와 비밀번호

`isLoggingIn`
로그인 요청 진행 여부

`accessToken`
발급받은 JWT

`currentUser`
현재 로그인한 회원

로그인 전

아이디·비밀번호 입력 폼
요청 중에는 로그인 버튼 비활성화

/login
↓
/me

로그인 후

회원 이름과 로그아웃 버튼
토큰과 회원 정보가 함께 있어야 표시

로그아웃하면 accessToken과 currentUser를 함께 초기화

로그인 화면: App 내부 선언

src/App.jsx

App 내부 선언, 이전 단계 18줄 뒤에 추가

```
});
const [wasSubmitted, setWasSubmitted] = useState(false);

 const [loginForm, setLoginForm] = useState({ userid: "", password: "" });
 const [accessToken, setAccessToken] = useState(null);
 const [currentUser, setCurrentUser] = useState(null);
 const [loginError, setLoginError] = useState("");
 const [isLoggingIn, setIsLoggingIn] = useState(false);


const featuredProduct = products[0];
```

로그인 화면: App 내부 선언 (1/4)

src/App.jsx

App 내부 선언, 이전 단계 121줄 뒤에 추가. 1/4

```
}
function handleLoginChange(event) {
  const { name, value } = event.target;
  setLoginForm((current) => ({ ...current, [name]: value }));
}

async function handleLoginSubmit(event) {
  event.preventDefault();
  setLoginError("");

  if (!loginForm.userid.trim() || !loginForm.password) {
    setLoginError("아이디와 비밀번호를 입력해 주세요.");
    return;
  }

  setIsLoggingIn(true);
  try {

   const response = await fetch("http://127.0.0.1:8000/login", {
     method: "POST",
```

로그인 화면: App 내부 선언 (2/4)

src/App.jsx

App 내부 선언, 이전 단계 121줄 뒤에 추가. 2/4

```
    setIsLoggingIn(true);
    try {
      const response = await fetch("http://127.0.0.1:8000/login", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(loginForm),
      });

      if (!response.ok) {
        throw new Error(
          response.status === 401
            ? "아이디 또는 비밀번호가 올바르지 않습니다."
            : `로그인 요청 실패 (${response.status})`,
        );
      }

      const { access_token: token } = await response.json();
      const meResponse = await fetch("http://127.0.0.1:8000/me", {

       headers: { Authorization: `Bearer ${token}` },
      });
```

로그인 화면: App 내부 선언 (3/4)

src/App.jsx

App 내부 선언, 이전 단계 121줄 뒤에 추가. 3/4

```
      const { access_token: token } = await response.json();
      const meResponse = await fetch("http://127.0.0.1:8000/me", {
        headers: { Authorization: `Bearer ${token}` },
      });

      if (!meResponse.ok) {
        throw new Error("로그인 상태를 확인하지 못했습니다.");
      }

      const user = await meResponse.json();
      setAccessToken(token);
      setCurrentUser(user);
      setLoginForm((current) => ({ ...current, password: "" }));
    } catch (error) {
      setAccessToken(null);
      setCurrentUser(null);
      setLoginError(error.message);
      setLoginForm((current) => ({ ...current, password: "" }));

    } finally {
      setIsLoggingIn(false);
```

로그인 화면: App 내부 선언 (4/4)

src/App.jsx

App 내부 선언, 이전 단계 121줄 뒤에 추가. 4/4

```
      setLoginError(error.message);
      setLoginForm((current) => ({ ...current, password: "" }));
    } finally {
      setIsLoggingIn(false);
    }
  }

  function handleLogout() {
    setAccessToken(null);
    setCurrentUser(null);
    setLoginError("");
  }


  const cartCount = cartItems.reduce((count, item) => count + item.quantity, 0);
  const cartTotal = cartItems.reduce((total, item) => {
```

로그인 화면: App의 return JSX (1/3)

src/App.jsx

App의 return JSX, 이전 단계 143줄 뒤에 추가. 1/3

```
        <span>취향에 맞는 물건을 찾아보세요.</span>
      </header>

      <section className="auth-section" aria-labelledby="auth-title">
        <h2 id="auth-title">고객 로그인</h2>
        {accessToken && currentUser ? (
          <div className="auth-signed-in">
            <p>
              <strong>{currentUser.name}</strong>님, 환영합니다.
            </p>
            <button type="button" onClick={handleLogout}>
              로그아웃
            </button>
          </div>
        ) : (
          <form className="auth-form" noValidate onSubmit={handleLoginSubmit}>
            <div>
              <label htmlFor="login-userid">아이디</label>

              <input
                id="login-userid"
```

로그인 화면: App의 return JSX (2/3)

src/App.jsx

App의 return JSX, 이전 단계 143줄 뒤에 추가. 2/3

```
          <div>
            <label htmlFor="login-userid">아이디</label>
              <input
                id="login-userid"
                name="userid"
                autoComplete="username"
                value={loginForm.userid}
                onChange={handleLoginChange}
              />
            </div>
            <div>
              <label htmlFor="login-password">비밀번호</label>
              <input
                id="login-password"
                name="password"
                type="password"
                autoComplete="current-password"
                value={loginForm.password}

              onChange={handleLoginChange}
            />
```

로그인 화면: App의 return JSX (3/3)

src/App.jsx

App의 return JSX, 이전 단계 143줄 뒤에 추가. 3/3

```
              autoComplete="current-password"
              value={loginForm.password}
                 onChange={handleLoginChange}
               />
             </div>
             <button type="submit" disabled={isLoggingIn}>
               {isLoggingIn ? "확인 중..." : "로그인"}
             </button>
             {loginError && (
               <p className="form-error" role="alert">
                 {loginError}
               </p>
             )}
           </form>
         )}
       </section>


     <section className="featured-section">
```

로그인 화면 확인

01 **실패**

잘못된 비밀번호로 오류 안내, 비밀번호 입력 초기화

02 **성공**

회원 이름, 로그아웃 버튼 표시

03 **로그아웃과 새로고침**

토큰과 고객 정보가 사라지는지 확인

응용 실습 1: 환영 문구

로그인 상태에 따라 환영 문구와 색을 바꾸기

성공 기준: 로그인 전 안내 → 로그인 후 회원 이름 → 로그아웃 후 안내 복원

힌트

1. src/components/WelcomeMessage.jsx에 새 컴포넌트 작성
2. App의 currentUser를 user Props로 전달
3. user 유무로 문구와 클래스를 선택

선택 심화

환영 문구와 배경색을 바꾸고 로그인 전후 결과를 비교

환영 문구 정답: 새 파일

src/components/WelcomeMessage.jsx

새 파일 생성 후 전체 입력

```
function WelcomeMessage({ user }) {
  return (
    <p className={user ? "welcome-message signed-in" : "welcome-message"}>
      {user
        ? `${user.name}님, 오늘의 상점에 다시 오셨네요!`
        : "로그인하고 나만의 주문을 시작해 보세요."}
    </p>
  );
}

export default WelcomeMessage;
```

환영 문구 정답: 두 위치 수정

src/App.jsx

각 위치 안내에 따라 해당 코드만 추가하거나 교체

import 문, 이전 단계 2줄 뒤에 추가

```
import "./App.css";
import WelcomeMessage from "./components/WelcomeMessage";

import ProductCard from "./components/ProductCard";
```

로그인 영역 JSX, 이전 단계 210–218줄을 아래 코드로 교체

```
      <section className="auth-section" aria-labelledby="auth-title">
        <h2 id="auth-title">고객 로그인</h2>
        <WelcomeMessage user={accessToken ? currentUser : null} />
        {accessToken && currentUser ? (
          <div className="auth-signed-in">
            <button type="button" onClick={handleLogout}>
              로그아웃

            </button>
```

환영 문구 실행 결과

01 **회원 표시**

기존 currentUser의 이름 사용

02 **조건 변경**

로그아웃하면 기본 안내 복원

2교시 핵심 정리

01 **로그인 요청**

입력값을 보내고 JWT와 회원 정보를 상태에 저장

02 **로그인 화면**

상태에 따라 폼, 회원 이름과 로그아웃 표시

03 **로그아웃과 새로고침**

메모리에 있던 인증 상태가 사라지면 다시 로그인

인증된 고객의 주문 저장

미션 - 주문과 상품별 판매 행을 저장하고 재고를 차감한다.

01 **완성 화면**

서버가 보낸 주문번호와 총액

02 **시간**

DB 7분 → 검증 8분 → API 15분 → 화면·확인 15분 → 응용 5분

주문과 판매 행의 관계

01

customers

한 고객

→

02

orders

한 번의 주문

→

03

sales

담은 상품별 행

→

04

products

상품과 재고

누가 최종 값을 결정하나요

고객 ID	요청 본문이 아닌 JWT에서 확인
가격과 총액	화면 미리 보기와 별개로 DB 가격으로 계산
수량과 입력	Pydantic과 서버 코드에서 검증
재고	서버가 DB 재고를 확인하고 차감

한 트랜잭션으로 저장

01

검사

입력, 중복 ID, 가격·재고

→

02

주문 번호

orders 1행 생성

→

03

상품별 저장

재고 차감과 sales

→

04

결과

모두 성공하면 커밋, 실패하면 롤백

잠금과 수량 제한

01 **수량과 상품 종류**

상품당 1~20개, 주문당 1~20종

02 **같은 상품 중복**

한 요청에서 같은 상품 ID가 반복되면 거절합니다.

03 **동시 주문**

상품 ID 순서로 FOR UPDATE 잠금을 얻고 재고를 확인합니다.

주문 API: 서버 코드

backend/main.py

서버 코드, 이전 단계 6–10줄을 아래 코드로 교체

```
from fastapi import Depends, HTTPException
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from jwt.exceptions import InvalidTokenError
from pwdlib import PasswordHash
from pydantic import BaseModel, Field
from decimal import Decimal
from fastapi.middleware.cors import CORSMiddleware


import os
import psycopg
```

주문 API: 서버 코드

backend/main.py

서버 코드, 이전 단계 79줄 뒤에 추가

```
    ]

class OrderItemIn(BaseModel):
    product_id: int = Field(gt=0)
    quantity: int = Field(gt=0, le=20)

class OrderIn(BaseModel):
    name: str = Field(min_length=1, max_length=50)
    email: str = Field(
        min_length=3,
        max_length=150,
        pattern=r"^[^\s@]+@[^\s@]+\.[^\s@]+$",
    )
    items: list[OrderItemIn] = Field(min_length=1, max_length=20)


@app.post("/login")
```

주문 API: 서버 코드 (1/6)

backend/main.py

서버 코드, 이전 단계 152줄 뒤에 추가. 1/6

```
def read_me(customer=Depends(get_current_customer)):
    return customer

@app.post("/orders", status_code=201)
def create_order(body: OrderIn, customer=Depends(get_current_customer)):
    contact_name = body.name.strip()
    if not contact_name:
        raise HTTPException(status_code=422, detail="주문자 이름을 입력해 주세요.")

    ids = [item.product_id for item in body.items]
    if len(ids) != len(set(ids)):
        raise HTTPException(status_code=422, detail="같은 상품이 중복되었습니다.")

    lines = []
    total = Decimal("0.00")
    sale_ids = []

    with get_connection() as connection:

        with connection.cursor() as cursor:
            for item in sorted(body.items, key=lambda value: value.product_id):
```

주문 API: 서버 코드 (2/6)

backend/main.py

서버 코드, 이전 단계 152줄 뒤에 추가. 2/6

```
    with get_connection() as connection:
        with connection.cursor() as cursor:
            for item in sorted(body.items, key=lambda value: value.product_id):
                cursor.execute(
                    """
                    SELECT product_id, name, price, stock_quantity
                    FROM public.products
                    WHERE product_id = %s
                    FOR UPDATE
                    """,
                    (item.product_id,),
                )
                product = cursor.fetchone()

                if product is None:
                    raise HTTPException(status_code=404, detail="상품을 찾을 수 없습니다.")
                if product["price"] < 0:

                    raise HTTPException(status_code=422, detail="상품 가격을 확인해 주세요.")
                if product["stock_quantity"] < item.quantity:
```

주문 API: 서버 코드 (3/6)

backend/main.py

서버 코드, 이전 단계 152줄 뒤에 추가. 3/6

```
            raise HTTPException(status_code=404, detail="상품을 찾을 수 없습니다.")
        if product["price"] < 0:
            raise HTTPException(status_code=422, detail="상품 가격을 확인해 주세요.")
        if product["stock_quantity"] < item.quantity:
            raise HTTPException(
                status_code=409,
                detail=f"{product['name']}의 재고가 부족합니다.",
            )

        line_total = product["price"] * item.quantity
        if line_total > Decimal("99999999.99"):
            raise HTTPException(status_code=422, detail="상품 금액 범위를 초과했습니다.")
        total += line_total
        lines.append((item, product, line_total))

    cursor.execute(
        """
        INSERT INTO public.orders (customer_id, contact_name, contact_email)

        VALUES (%s, %s, %s)
        RETURNING order_id
```

주문 API: 서버 코드 (4/6)

backend/main.py

서버 코드, 이전 단계 152줄 뒤에 추가. 4/6

```
            """
            INSERT INTO public.orders (customer_id, contact_name, contact_email)
                VALUES (%s, %s, %s)
                RETURNING order_id
                """,
                (customer["customer_id"], contact_name, body.email),
            )
            order_id = cursor.fetchone()["order_id"]

            for item, product, line_total in lines:
                cursor.execute(
                    """
                    UPDATE public.products
                    SET stock_quantity = stock_quantity - %s
                    WHERE product_id = %s AND stock_quantity >= %s
                    RETURNING stock_quantity
                    """,
                    (item.quantity, item.product_id, item.quantity),

            )
            if cursor.fetchone() is None:
```

주문 API: 서버 코드 (5/6)

backend/main.py

서버 코드, 이전 단계 152줄 뒤에 추가. 5/6

```
            """,
            (item.quantity, item.product_id, item.quantity),
            )
            if cursor.fetchone() is None:
                raise HTTPException(status_code=409, detail="재고가 부족합니다.")

            cursor.execute(
                """
                INSERT INTO public.sales
                    (order_id, customer_id, product_id, quantity, total_price)
                VALUES (%s, %s, %s, %s, %s)
                RETURNING sale_id
                """,
                (
                    order_id,
                    customer["customer_id"],
                    item.product_id,
                    item.quantity,

                line_total,
            ),
```

주문 API: 서버 코드 (6/6)

backend/main.py

서버 코드, 이전 단계 152줄 뒤에 추가. 6/6

```
                    item.product_id,
                    item.quantity,
                        line_total,
                    ),
                )
                sale_ids.append(cursor.fetchone()["sale_id"])

    return {
        "order_id": order_id,
        "sale_ids": sale_ids,
        "total_price": float(total),
    }
```

주문 응답과 오류

201	주문번호, 판매 행 ID 목록, 서버 총액
401	인증 필요 또는 만료, 다시 로그인
409	재고 부족, 주문 전체 저장 실패
422와 404	잘못된 입력·중복 상품 또는 없는 상품

주문 상태와 화면

01 **isSubmittingOrder**

저장 중 버튼을 비활성화합니다.

02 **orderError**

실패 원인을 표시하고 장바구니는 유지합니다.

03 **orderResult**

서버 응답을 표시하고 성공하면 장바구니를 비웁니다.

주문 화면: App 내부 선언

src/App.jsx

App 내부 선언, 이전 단계 24줄 뒤에 추가

```
  const [loginError, setLoginError] = useState("");
  const [isLoggingIn, setIsLoggingIn] = useState(false);

  const [isSubmittingOrder, setIsSubmittingOrder] = useState(false);
  const [orderError, setOrderError] = useState("");
  const [orderResult, setOrderResult] = useState(null);


  const featuredProduct = products[0];
```

주문 화면: handleOrderChange 함수 (1/4)

src/App.jsx

handleOrderChange 함수, 이전 단계 121–127줄을 아래 코드로 교체. 1/4

```
  const { name, value } = event.target;
  setOrderForm((currentForm) => ({ ...currentForm, [name]: value }));
}

async function handleOrderSubmit(event) {
  event.preventDefault();
  setWasSubmitted(true);
  setOrderError("");
  setOrderResult(null);

  if (!isOrderValid) return;
  if (!accessToken) {
    setOrderError("주문하려면 먼저 로그인해 주세요.");
    return;
  }

  setIsSubmittingOrder(true);
  try {

   const response = await fetch("http://127.0.0.1:8000/orders", {
     method: "POST",
```

주문 화면: handleOrderChange 함수 (2/4)

src/App.jsx

handleOrderChange 함수, 이전 단계 121–127줄을 아래 코드로 교체. 2/4

```
    setIsSubmittingOrder(true);
    try {
      const response = await fetch("http://127.0.0.1:8000/orders", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
          Authorization: `Bearer ${accessToken}`,
        },
        body: JSON.stringify({
          name: orderForm.name.trim(),
          email: orderForm.email.trim(),
          items: cartItems.map((item) => ({
            product_id: item.productId,
            quantity: item.quantity,
          })),
        }),
      });


      const data = await response.json().catch(() => ({}));
      if (response.status === 401) {
```

주문 화면: handleOrderChange 함수 (3/4)

src/App.jsx

handleOrderChange 함수, 이전 단계 121–127줄을 아래 코드로 교체. 3/4

```
    });
    const data = await response.json().catch(() => ({}));
    if (response.status === 401) {
      setAccessToken(null);
      setCurrentUser(null);
      throw new Error("로그인이 만료되었습니다. 다시 로그인해 주세요.");
    }
    if (!response.ok) {
      throw new Error(
        typeof data.detail === "string"
          ? data.detail
          : `주문을 저장하지 못했습니다 (${response.status}).`,
      );
    }

    setOrderResult(data);
    setCartItems([]);

    setWasSubmitted(false);
  } catch (error) {
```

주문 화면: handleOrderChange 함수 (4/4)

src/App.jsx

handleOrderChange 함수, 이전 단계 121–127줄을 아래 코드로 교체. 4/4

```
      setOrderResult(data);
      setCartItems([]);
      setWasSubmitted(false);
    } catch (error) {
      setOrderError(error.message || "서버 연결을 확인해 주세요.");
    } finally {
      setIsSubmittingOrder(false);
    }
  }


function handleLoginChange(event) {
  const { name, value } = event.target;
```

주문 화면: 장바구니 JSX

src/App.jsx

장바구니 JSX, 이전 단계 389–393줄을 아래 코드로 교체

```
          </>
        )}
      </section>
      <section className="order-section" aria-labelledby="order-title">
        <h2 id="order-title">주문서</h2>
        <div className="order-layout">
          <form className="order-form" noValidate onSubmit={handleOrderSubmit}>

            <h3>주문자 정보</h3>
            <label htmlFor="order-name">이름</label>
```

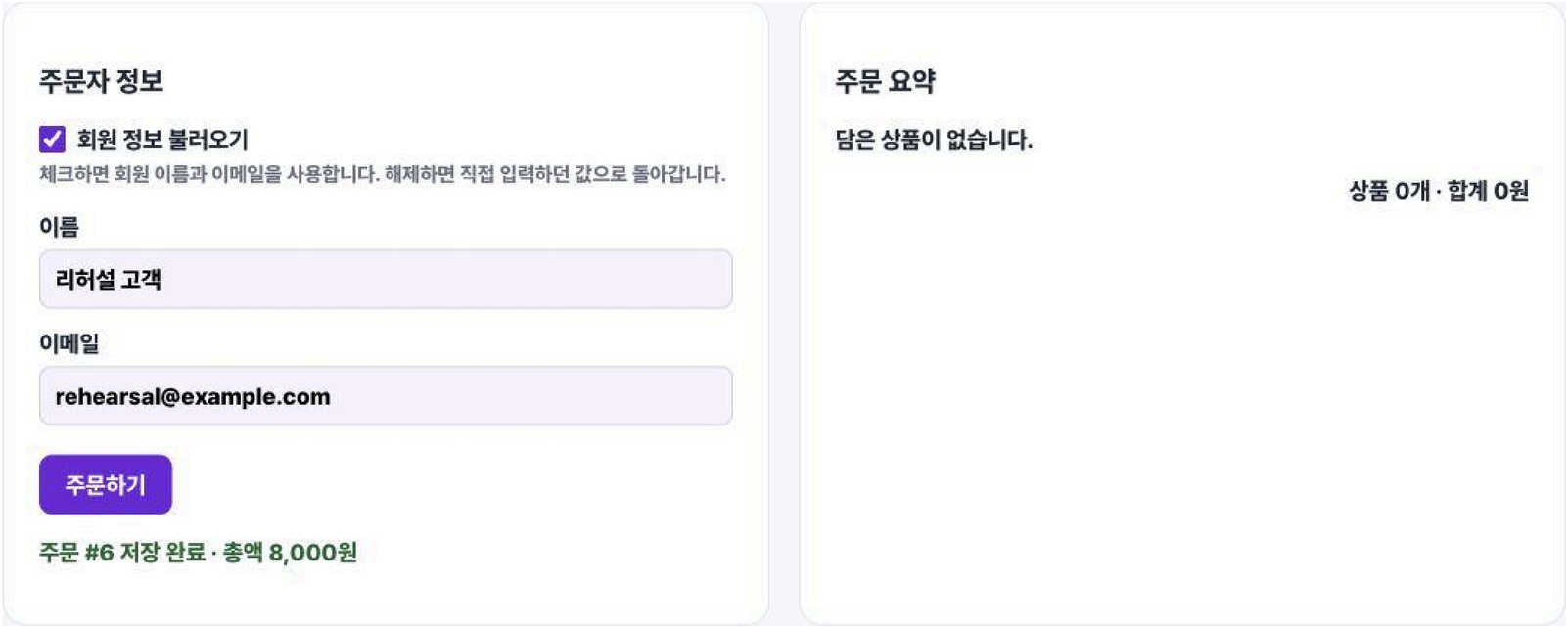
주문 화면: 주문서 JSX

src/App.jsx

주문서 JSX, 이전 단계 427–434줄을 아래 코드로 교체

```
            <p className="form-error" role="alert">
              {orderErrors.cart}
            </p>
          )}
          <button type="submit" disabled={isSubmittingOrder}>
            {isSubmittingOrder ? "주문 저장 중..." : "주문하기"}
          </button>
          {orderError && (
            <p className="form-error" role="alert">
              {orderError}
            </p>
          )}
          {orderResult && (
            <p className="form-success" role="status">
              주문 #{orderResult.order_id} 저장 완료 · 총액{" "}
              {orderResult.total_price.toLocaleString()}원
            </p>
          )}

        </form>
```

주문 저장 실행 결과



01 **주문 결과**

주문번호와 서버 총액은 남음

02 **현재 장바구니**

주문 성공 후 비워져 합계 0원

응용 실습 2: 주문 성공 카드

서버가 보낸 주문번호와 총액을 카드로 표시

성공 기준: 주문 성공 후 장바구니가 비어도 서버 총액은 카드에 유지

힌트

1. src/components/OrderSuccessCard.jsx에 새 컴포넌트 작성
2. App의 orderResult를 Props로 전달
3. 주문번호와 서버 총액 표시, 장바구니에서 재계산하지 않기

선택 심화

카드 제목, 강조 색과 정보 배치를 바꾸고 주문 결과 비교

주문 카드 정답: 새 파일

src/components/OrderSuccessCard.jsx

새 파일 생성 후 전체 입력

```jsx
function OrderSuccessCard({ result }) {
  if (!result) return null;

  return (
    <section className="order-success-card" role="status">
      <h3>주문이 저장되었습니다</h3>
      <p>주문번호: #{result.order_id}</p>
      <p>서버 총액: {result.total_price.toLocaleString()}원</p>
      <p>실제 결제는 포함하지 않은 실습 주문입니다.</p>
    </section>
  );
}

export default OrderSuccessCard;
```

주문 카드 정답: 두 위치 수정

src/App.jsx

각 위치 안내에 따라 해당 코드만 추가하거나 교체

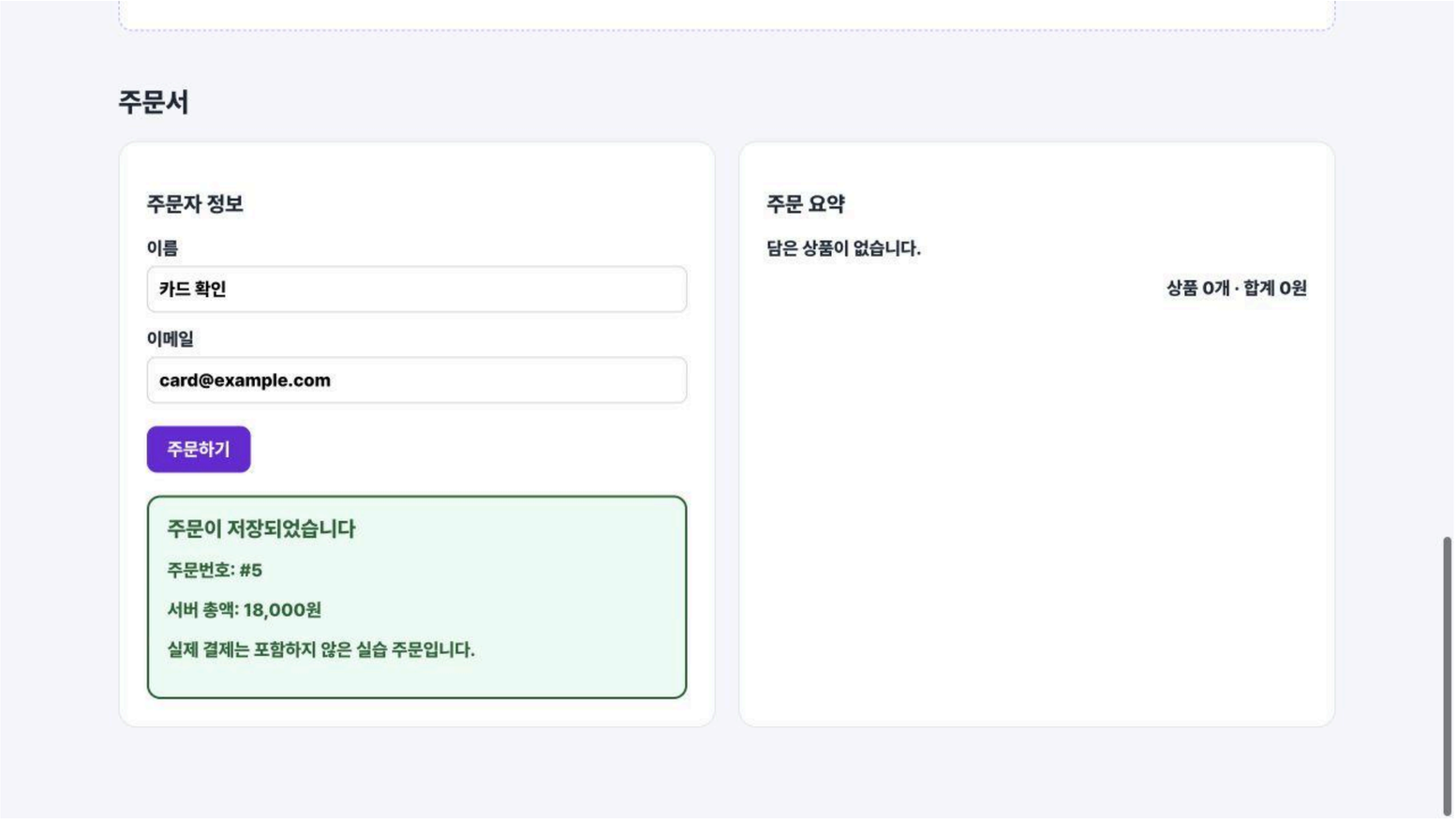
import 문, 이전 단계 2줄 뒤에 추가

```
import "./App.css";
import OrderSuccessCard from "./components/OrderSuccessCard";

import ProductCard from "./components/ProductCard";
```

주문서 JSX, 이전 단계 488–497줄을 아래 코드로 교체

```
              {orderError}
              </p>
            )}
            <OrderSuccessCard result={orderResult} />
          </form>


        <div className="order-summary">
```

주문 카드 실행 결과



01 **카드 금액**

서버가 응답한 18,000원

02 **오른쪽 요약**

현재 장바구니는 비어 있음

3교시 핵심 정리

01 **서버 검증**

고객, 가격, 입력과 재고의 최종 기준

02 **트랜잭션**

주문·판매·재고를 함께 성공하거나 되돌림

03 **성공 화면**

서버 주문 결과와 현재 장바구니를 구분

화면 이동과 회원 정보 연결

미션 - 주소별 화면으로 나누고 회원 정보로 주문한다.

01 **완성 화면**

/, /login, /cart와 회원 정보 주문서

02 **시간**

라우팅 5분 → 메뉴 6분 → 화면 12분 → 이동 4분
회원 정보 12분 → 응용 6분 → 정리 5분

주소별 화면과 공통 상태

/	추천 상품, 상품 목록, 검색과 필터
/login	로그인 폼 또는 회원 이름과 로그아웃
/cart	장바구니, 로그인 안내 또는 주문서
App	상품·장바구니·인증·주문 상태를 계속 소유

이동 경로

01
상품
검색하고 담기

→

02
장바구니
담은 상품 확인

→

03
로그인
/me까지 성공

→

04
장바구니
회원 정보와 주문

React Router 설치

Windows CMD와 Mac 공통

package.json이 있는 프로젝트 폴더에서 실행

```
npm install react-router
```

메뉴와 함수에서 이동

BrowserRouter	App에 주소 정보를 제공
Routes와 Route	현재 경로에 해당하는 화면 선택
NavLink	메뉴 클릭과 현재 메뉴 표시
useNavigate	로그인 성공 후 /cart로 이동

화면 연결: React 시작점

src/main.jsx

React 시작점 전체 교체

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import { BrowserRouter } from "react-router";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
);
```

화면 연결

src/components/ShopNav.jsx

새 파일 생성 후 전체 입력

```
import { NavLink } from "react-router";

function ShopNav({ cartCount, currentUser }) {
  return (
    <nav className="shop-nav" aria-label="상점 메뉴">
      <NavLink to="/" end>
        상품
      </NavLink>
      <NavLink to="/cart">장바구니 ({cartCount})</NavLink>
      <NavLink to="/login">{currentUser ? `${currentUser.name}님` : "로그인"}</NavLink>
    </nav>
  );
}

export default ShopNav;
```

화면 연결

src/App.jsx

각 위치 안내에 따라 해당 코드만 추가하거나 교체

import 문, 이전 단계 1-8줄을 아래 코드로 교체

```
import { useEffect, useState } from "react";
import "./App.css";
import { Routes, Route, Link, useNavigate } from "react-router";
import ShopNav from "./components/ShopNav";
import ProductCard from "./components/ProductCard";

function App() {
  const shopName = "오늘의 상점";
  const navigate = useNavigate();
  const [products, setProducts] = useState([]);
  const [isLoading, setIsLoading] = useState(true);

  const [productError, setProductError] = useState("");
```

handleLoginSubmit 함수, 이전 단계 222줄 뒤에 추가

```
      setCurrentUser(user);
       navigate("/cart");

      setLoginForm((current) => ({ ...current, password: "" }));
```

화면 연결: handleLogout 함수

src/App.jsx

handleLogout 함수, 이전 단계 237줄 뒤에 추가

```
    setCurrentUser(null);
    setLoginError("");
     navigate("/login");

  }
```

화면 연결: App의 return JSX

src/App.jsx

App의 return JSX, 이전 단계 260–264줄을 아래 코드로 교체

```
        <p>작은 발견이 있는 쇼핑</p>
        <h1>{shopName}</h1>
          <span>취향에 맞는 물건을 찾아보세요.</span>
        </header>
        <ShopNav cartCount={cartCount} currentUser={currentUser} />
        <Routes>
          <Route
            path="/login"
            element={
              <section className="auth-section" aria-labelledby="auth-title">
                <h2 id="auth-title">고객 로그인</h2>

              {accessToken && currentUser ? (
                <div className="auth-signed-in">
```

화면 연결: 로그인 영역 JSX

src/App.jsx

로그인 영역 JSX, 이전 단계 306–310줄을 아래 코드로 교체

```
              )}
            </form>
            )}
          </section>
        }
      />
      <Route
        path="/"
        element={
          <>
            <section className="featured-section">
              <h2>오늘의 추천 상품</h2>

            {featuredProduct ? (
              <article className="featured-product">
```

화면 연결: 추천 상품 JSX

src/App.jsx

추천 상품 JSX, 이전 단계 320–324줄을 아래 코드로 교체

```
                  <button type="button">상품 보러 가기</button>
                </article>
                ) : (
                  <p className="empty-state">
                    {isLoading
                      ? "추천 상품을 불러오는 중입니다."
                      : "추천 상품이 없습니다."}
                  </p>
                )}

            </section>
```

화면 연결: 전체 상품 JSX

src/App.jsx

전체 상품 JSX, 이전 단계 369–373줄을 아래 코드로 교체

```
                  상품을 불러오지 못했습니다: {productError}
                </p>
              ) : (
                <>
                  <p className="result-count">
                    검색 결과: {filteredProducts.length}개
                  </p>
                  {filteredProducts.length > 0 ? (
                    <div className="product-grid">

                  {filteredProducts.map((product) => (
                    <ProductCard
```

화면 연결: 장바구니 JSX

src/App.jsx

장바구니 JSX, 이전 단계 391줄 뒤에 추가

```
              )}
            </section>
          </>
        }
      />
      <Route
        path="/cart"
        element={
          <>

            <section className="cart-section" aria-labelledby="cart-title">
              <h2 id="cart-title">장바구니 ({cartCount}개)</h2>
```

화면 연결: 장바구니 JSX

src/App.jsx

장바구니 JSX, 이전 단계 402–406줄을 아래 코드로 교체

```
            ) : (
              <>
                <ul className="cart-list">
                  {cartItems.map((item) => {
                    const product = products.find(
                      (entry) => entry.id === item.productId,
                    );
                    if (!product) return null;
                    return (

                     <li className="cart-item" key={item.productId}>
                      <div>
```

화면 연결: 장바구니 JSX

src/App.jsx

장바구니 JSX, 이전 단계 413–417줄을 아래 코드로 교체

```
                  {product.price.toLocaleString()}원 × {item.quantity}
                </p>
                </div>
                <div className="quantity-controls">
                  <button
                    type="button"
                    onClick={() => removeFromCart(product.id)}
                  >
                    삭제
                  </button>


                <button
```

화면 연결: 장바구니 JSX

src/App.jsx

장바구니 JSX, 이전 단계 441–448줄을 아래 코드로 교체

```
                  <p className="cart-total">합계: {cartTotal.toLocaleString()}원</p>
                </>
              )}
            </section>
            {accessToken && currentUser ? (
              <section className="order-section" aria-labelledby="order-title">
                <h2 id="order-title">주문서</h2>
                <div className="order-layout">
                  <form
                    className="order-form"
                    noValidate
                    onSubmit={handleOrderSubmit}
                  >
                    <h3>주문자 정보</h3>
                    <label htmlFor="order-name">이름</label>

                  <input
                    id="order-name"
```

화면 연결: 주문서 JSX

src/App.jsx

주문서 JSX, 이전 단계 503–507줄을 아래 코드로 교체

```
                  <p>담은 상품이 없습니다.</p>
                ) : (
                  <ul>
                    {cartItems.map((item) => {
                      const product = products.find(
                        (entry) => entry.id === item.productId,
                      );
                      if (!product) return null;
                      return (

                        <li key={item.productId}>
                          <span>
```

화면 연결: 주문서 JSX

src/App.jsx

주문서 JSX, 이전 단계 510–514줄을 아래 코드로 교체

```
                        <li key={item.productId}>
                          <span>
                                  {product.name} × {item.quantity}
                                </span>
                                <span>
                                  {(product.price * item.quantity).toLocaleString()}원
                                </span>
                              </li>
                            );

                      })}
                    </ul>
```

화면 연결: 주문서 JSX (1/2)

src/App.jsx

주문서 JSX, 이전 단계 523줄 뒤에 추가. 1/2

```
              </div>
            </section>
            ) : (
              <p className="empty-state">
                주문하려면 <Link to="/login">로그인</Link>해 주세요.
              </p>
            )}
          </>
        }
      />
      <Route
        path="*"
        element={
          <p className="empty-state">
            없는 화면입니다. <Link to="/">상품 보기</Link>
          </p>
        }
      />

    </Routes>
  </main>
```

화면 연결: 주문서 JSX (2/2)

src/App.jsx

주문서 JSX, 이전 단계 523줄 뒤에 추가. 2/2

```
          }
        />
      </Routes>

    </main>
  );
```

로그인 안내와 서버 인증

01 **로그인 전 /cart**

장바구니는 보이고 주문서 대신 로그인 안내를 표시합니다.

02 **로그인 성공**

/me 응답을 확인한 후 /cart로 이동합니다.

03 **주문 권한**

화면 조건과 별개로 서버가 JWT를 다시 검증합니다.

회원 정보는 표시할 값으로 계산

01 **직접 입력**

orderForm은 사용자가 적은 값을 기억합니다.

02 **체크 상태**

useMemberInfo는 회원 정보를 쓸지 기억합니다.

03 **화면의 값**

orderName과 orderEmail은 두 정보 중 표시할 값을 고릅니다.

회원 정보 연결

src/App.jsx

각 위치 안내에 따라 해당 코드만 추가하거나 교체

App 내부 선언, 이전 단계 20줄 뒤에 추가

```
});
const [useMemberInfo, setUseMemberInfo] = useState(false);

const [wasSubmitted, setWasSubmitted] = useState(false);
```

handleOrderSubmit 함수, 이전 단계 149–154줄을 아래 코드로 교체

```
        Authorization: `Bearer ${accessToken}`,
        },
        body: JSON.stringify({
          name: orderName.trim(),
          email: orderEmail.trim(),
          items: cartItems.map((item) => ({
            product_id: item.productId,

          quantity: item.quantity,
```

회원 정보 연결

src/App.jsx

각 위치 안내에 따라 해당 코드만 추가하거나 교체

handleOrderSubmit 함수, 이전 단계 163줄 뒤에 추가

```
      setCurrentUser(null);
        setUseMemberInfo(false);
        setOrderForm({ name: "", email: "" });
        setOrderResult(null);
        setWasSubmitted(false);

      throw new Error("로그인이 만료되었습니다. 다시 로그인해 주세요.");
```

handleLoginSubmit 함수, 이전 단계 230줄 뒤에 추가

```
    setCurrentUser(null);
     setUseMemberInfo(false);
     setOrderForm({ name: "", email: "" });
     setOrderResult(null);
     setWasSubmitted(false);

    setLoginError(error.message);
```

회원 정보 연결: handleLogout 함수

src/App.jsx

handleLogout 함수, 이전 단계 240줄 뒤에 추가

```
    setAccessToken(null);
    setCurrentUser(null);
     setUseMemberInfo(false);
     setOrderForm({ name: "", email: "" });
     setOrderResult(null);
     setWasSubmitted(false);

    setLoginError("");
    navigate("/login");
```

회원 정보 연결: App 내부 선언

src/App.jsx

App 내부 선언, 이전 단계 249–255줄을 아래 코드로 교체

```
  const product = products.find((entry) => entry.id === item.productId);
  return total + (product?.price ?? 0) * item.quantity;
}, 0);

const orderName = useMemberInfo && currentUser ? currentUser.name : orderForm.name;
const orderEmail = useMemberInfo && currentUser ? currentUser.email : orderForm.email;

const orderErrors = {
  name: orderName.trim() ? "" : "이름을 입력해 주세요.",
  email: /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(orderEmail.trim())
    ? ""
    : "올바른 이메일을 입력해 주세요.",

  cart: cartCount > 0 ? "" : "장바구니에 상품을 담아 주세요.",
};
```

회원 정보 연결: 주문서 JSX (1/2)

src/App.jsx

주문서 JSX, 이전 단계 481–489줄을 아래 코드로 교체. 1/2

```
                noValidate
                onSubmit={handleOrderSubmit}
                >
                  <h3>주문자 정보</h3>
                  <label className="member-info-option" htmlFor="use-member-info">
                    <input
                      id="use-member-info"
                      type="checkbox"
                      checked={useMemberInfo}
                      onChange={(event) => setUseMemberInfo(event.target.checked)}
                    />
                    회원 정보 불러오기
                  </label>
                  <p className="member-info-help">
                    체크하면 회원 이름과 이메일을 사용합니다. 해제하면 직접 입력하던
                    값으로 돌아갑니다.
                  </p>
                  <label htmlFor="order-name">이름</label>

                <input
                  id="order-name"
```

회원 정보 연결: 주문서 JSX (2/2)

src/App.jsx

주문서 JSX, 이전 단계 481–489줄을 아래 코드로 교체. 2/2

```
                    </p>
                    <label htmlFor="order-name">이름</label>
                        <input
                          id="order-name"
                          name="name"
                          value={orderName}
                          readOnly={useMemberInfo}
                          onChange={handleOrderChange}
                          aria-invalid={wasSubmitted && Boolean(orderErrors.name)}

                    />
                    {wasSubmitted && orderErrors.name && (
```

회원 정보 연결: 주문서 JSX

src/App.jsx

주문서 JSX, 이전 단계 500–504줄을 아래 코드로 교체

```
                    <input
                      id="order-email"
                           name="email"
                           type="email"
                           value={orderEmail}
                           readOnly={useMemberInfo}
                           onChange={handleOrderChange}
                           aria-invalid={wasSubmitted && Boolean(orderErrors.email)}

                    />
                    {wasSubmitted && orderErrors.email && (
```

회원 정보 실행 결과

01 **체크**

회원 이름과 이메일
읽기 전용으로 표시

02 **해제**

기존 직접 입력값으로 복원

회원 정보와 초기화 확인

체크·해제	회원 값 표시와 직접 입력값 복원
주문 요청	검사하는 값과 서버에 보내는 값이 화면과 일치
로그아웃·401	회원 선택과 주문자 입력 초기화
새로고침	메모리 로그인·장바구니 상태 초기화

응용 실습 3: 현재 메뉴 강조

현재 메뉴에 체크 표시와 강조 스타일 적용

성공 기준: 상품·장바구니·로그인, 뒤로가기에서도 현재 메뉴 하나만 강조

힌트

1. src/components/ShopNav.jsx의 기존 메뉴 수정
2. NavLink의 isActive와 active 클래스 사용
3. / 링크의 end와 App의 공유 상태 유지

선택 심화

표시 문자, 배경색과 밑줄을 바꾸고 메뉴 이동 결과 비교

현재 메뉴 정답: ShopNav 함수 (1/2)

src/components/ShopNav.jsx

ShopNav 함수 전체 교체. 1/2

```
import { NavLink } from "react-router";

function ShopNav({ cartCount, currentUser }) {
  return (
    <nav className="shop-nav" aria-label="상점 메뉴">
      <NavLink to="/" end>
        {({ isActive }) => `${isActive ? "✓ " : ""}상품`}
      </NavLink>
      <NavLink to="/cart">
        {({ isActive }) => `${isActive ? "✓ " : ""}장바구니 (${cartCount})`}
      </NavLink>
      <NavLink to="/login">
        {({ isActive }) =>
          `${isActive ? "✓ " : ""}${currentUser ? `${currentUser.name}님` : "로그인"}`
        }
      </NavLink>

    </nav>
  );
```

현재 메뉴 정답: ShopNav 함수 (2/2)

src/components/ShopNav.jsx

ShopNav 함수 전체 교체. 2/2

```
        }
      </NavLink>
    </nav>
  );
}

export default ShopNav;
```

현재 메뉴 실행 결과

01 **경로**

/cart에서 장바구니 메뉴 강조

02 **공유 정보**

메뉴는 기존 cartCount와 currentUser를 표시

4교시 핵심 정리

01 **라우팅**

주소별 부분 화면을 선택

02 **공유 상태**

App에 남겨 메뉴 이동 중 유지

03 **파생값**

체크 상태에 맞춰 주문자 표시값 계산